

数据工程与特征工程

数据与特征决定了机器学习模型的上限，算法只是逼近这个上限的手段。本文系统介绍数据管道架构、数据采集、Feature Store 等核心内容，覆盖从原始数据到模型可用特征的完整链路。

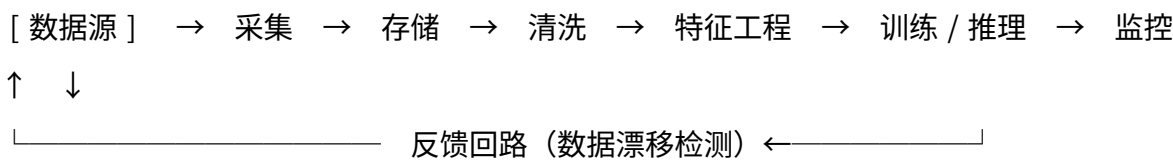
数据工程与特征工程

数据与特征决定了机器学习模型的上限，算法只是逼近这个上限的手段。本文系统介绍数据管道架构、数据采集、Feature Store 等核心内容，覆盖从原始数据到模型可用特征的完整链路。

- - -

一、数据管道架构

现代机器学习系统的数据管道通常包含以下阶段：



1 . 1 Lambda 与 Kappa 架构

架构 批处理层 流处理层 服务层 适用场景

Lambda Batch Layer Speed Layer Serving Layer 历史与现状
Kappa — 统一流处理 Serving Layer 仅流处理，架构简洁

1 . 2 常用组件栈

- 采集：Kafka、Pulsar、Flume、CDC (Debezium)
- 存储：HDFS、S3、Hive、Iceberg、Delta Lake、Hudi
- 计算：Spark、Flink、Ray、Dask
- 特征存储：Feast、Tecton、Hopsworks、AWS Feature Store

- - -

二、数据采集与清洗

2 . 1 数据采集方式

- 批式采集：定时拉取数据库、日志文件（S q o o p、D a t a X）
- 流式采集：实时订阅事件（K a f k a C o n n e c t、F l i n k C D C）
- 开源数据集：H u g g i n g F a c e D a t a s e t s、K a g g l e、U C I
- 合成数据：用于隐私保护与数据增强（C T G A N、D i f f u s i o n 模型）

2.2 数据清洗核心步骤

1. 缺失值处理

- 删除：缺失率 $> 50\%$ 或行 / 列缺失严重
- 均值 / 中位数 / 众数填充：数值型常用中位数，类别型用众数
- 模型填充：用 K N N、回归或迭代插补（I t e r a t i v e I m p u t e r）
- 标记缺失：增加 i s m i s s i n g 指示特征，保留 " 缺失 " 信号

2. 异常值检测

- 统计方法： 3σ 准则、I Q R（箱线图）
- 距离方法：K N N 距离、L O F（局部异常因子）
- 模型方法：I s o l a t i o n F o r e s t、O n e - C l a s s S V M、A u t o e n c o d e r

3. 重复值与冲突

- 完全重复：保留一条
- 主键冲突：根据业务规则取舍

4. 数据类型与一致性

- 统一日期格式、单位、编码
- 类别值大小写归一化（N e w Y o r k v s n e w y o r k）

```
import pandas as pd
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import IsolationForest

df = pd.readcsv("data.csv")
```

缺失值：数值型迭代插补 + 缺失指示

```
numcols = df.selectdtypes(include=np.number).columns
```

```
df[numcols + "isna"] = df[numcols].isna()
imp = IterativeImputer(randomstate=42)
df[numcols] = imp.fit_transform(df[numcols])
```

异常值：Isolation Forest

```
iso = IsolationForest(contamination=0.02, randoms
df["isoutlier"] = iso.fit_predict(df[numcols])
dfclean = df[df["isoutlier"] == 1].drop(columns=
- - -
```

三、特征类型与编码

3.1 数值特征

- 直接使用或经变换（见第四节）
- 大数值可分箱为类别特征，捕捉非线性

3.2 类别特征编码

方法 描述 适用场景

Label Encoding 整数映射 有序类别（如低 / 中 / 高）

One-Hot 独热向量 低基数无序类别

Target Encoding 用目标均值替换 高基数类别（如邮编）

Frequency Encoding 用频次替换 高基数类别

Embedding 学习低维向量 极高基数（如用户 ID、商品 ID）

Hashing 哈希到固定维度 流式场景、避免维护映射表

Target Encoding 防泄露：必须用 K 折交叉验证计算目标均值，或加入平滑项：

$$\hat{x}_k = \frac{\sum_{i \in k} y_i}{|k|} + \alpha \cdot \text{cd}$$

3.3 文本特征

- 传统方法：Bag-of-Words、TF-IDF、N-gram
- 主题模型：LDA、NMF
- 词向量：Word2Vec、GloVe、FastText

- 上下文向量：B E R T、S e n t e n c e - B E R T 嵌入（推荐用于现代任务）

3.4 时间特征

从时间戳提取：年、月、日、时、分、秒、星期、是否周末、是否节假日、季度、所属季节。还可构造：

- 周期性编码： $\sin(2\pi h/24)$, $\cos(2\pi h/24)$ （小时）
- 距重要日期差值：距春节天数、距促销开始秒数
- 滞后特征与滑动统计： x_{t-1} , $\text{text\{mean\}}(x_{t-7:t-1})$

3.5 图像特征

- 手工特征：H O G、S I F T、S U R F、L B P（传统 CV）
- 深度特征：C N N 中间层输出、C L I P 图像嵌入
- 数据增强：翻转、裁剪、色彩抖动、M i x u p、C u t M i x

- - -

四、特征变换

4.1 标准化与归一化

方法 公式 适用

StandardScaler $(x - \mu) / \sigma$ 服从正态分布，SVM / LR
 MinMaxScaler $(x - \min) / (\max - \min)$ 已知边界，神经网络
 MaxAbsScaler $x / \max(|x|)$ 稀疏数据
 RobustScaler $(x - \text{text\{median\}}) / \text{text\{IQR\}}$ 含异常值

4.2 非线性变换

- 对数变换： $x' = \log(1 + x)$ ，处理右偏分布（收入、点击量）
- Box-Cox： $x^{(\lambda)} = \frac{x^\lambda - 1}{\lambda}$
- Yeo-Johnson：支持 $x \leq 0$ ，更通用
- 分位数变换：映射到均匀或正态分布，对极端值稳健

4.3 分箱（Binning）

- 等宽分箱：按值域均分
- 等频分箱：每箱样本数相同
- 决策树分箱：用树的最优分裂点，自动捕捉与目标的关系
- WOE 分箱：信用评分中常用， $WOE = \ln(\text{好样本占比} / \text{坏样本占比})$
- - -

五、特征选择

5.1 Filter (过滤法)

基于统计指标独立评估每个特征，与模型无关。

- 方差阈值：删除方差过小的特征
- 相关系数：Pearson、Spearman、卡方检验
- 互信息： $I(X; Y) = -\sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$
- ANOVA F-value：分类任务中评估特征区分度

5.2 Wrapper (包装法)

以模型性能为评估指标，搜索最优特征子集。

- 递归特征消除 (RFE)：递归剔除最不重要的特征
- 前向 / 后向选择：贪心增删特征
- 计算成本高，适合小到中等规模数据

5.3 Embedded (嵌入法)

在模型训练过程中完成特征选择。

- L1 正则 (Lasso)：产生稀疏解，自动选择特征
- 树模型重要性：feature importances 基于 Gini 或信息增益
- SHAP 值：基于博弈论的特征贡献，统一解释

```
from sklearn.featureselection import (VarianceThreshold,
mutualinfoclassif, RFE)
from sklearn.linearmodel import LassoCV
from sklearn.ensemble import RandomForestClassifier
```

```
import shap
```

Filter: 方差阈值 + 互信息

```
Xsel = VarianceThreshold(threshold=0.01).fit_transform(X)
```

```
Xsel = SelectKBest(mutualinfo_classif, k=20).fit_transform(Xsel)
```

Embedded: Lasso

```
lasso = LassoCV(cv=5).fit(X, y)
```

```
selected = np.where(lasso.coef != 0)[0]
```

Wrapper: RFE

```
rfe = RFE(RandomForestClassifier(n_estimators=100))
```

```
rfe.fit(X, y)
```

SHAP 解释

```
explainer = shap.TreeExplainer(rfmodel)
```

```
shap_values = explainer.shap_values(Xtest)
```

```
shap.summary_plot(shap_values, Xtest)
```

- - -

六、特征构造与交叉特征

6.1 特征交叉

- 类别交叉: $city \times productcategory$ 捕捉局部模式
- 数值交互: $x_1 \times x_2$ 、 x_1 / x_2 、 $x_1 - x_2$
- FM / Deep FM: 自动学习二阶以上交叉, 无需人工枚举

6.2 业务特征构造

- 比率特征: 转化率、点击率
- 聚合特征: 按用户 / 商品维度统计均值、求和、计数
- 时序特征: 滞后值、滑动窗口、指数加权移动平均
- 图特征: 度中心性、PageRank、社区发现标签

6.3 自动特征构造

工具：Featuretools（深度特征合成DFS）、AutoFeat、TSFresh（时间序列）。自动生成大量候选特征后必须配合特征选择，否

- - -

七、数据增强技术

模态 增强方法

图像 翻转、旋转、裁剪、色彩抖动、Cutout、Mixup、CutMix、AutoAugment

文本 同义词替换、回译、EDA、上下文词插入、Masked LM 生成

音频 加噪、变速、SpecAugment（时间 / 频率掩码）

表格 SMOTE、ADASYN、CTGAN 生成、Mixup

信号 加噪声、时移、频域扰动

注意：增强应保持标签语义不变，验证集与测试集不可增强。

- - -

八、数据质量评估

8 . 1 数据质量维度

维度 含义 评估方式

完整性 缺失程度 缺失率统计

准确性 值是否真实正确 规则校验、抽样核查

一致性 多源数据是否一致 跨表对比、约束检查

时效性 数据是否最新 时间戳延迟监控

唯一性 是否存在重复 主键去重检测

合规性 是否符合隐私法规 PII 扫描、脱敏审计

8 . 2 数据漂移检测

模型上线后，特征分布与目标分布可能漂移。常用指标：

- PSI (Population Stability Index) : $PSI = \sum (p_i - q_i) \ln \frac{p_i}{q_i}$
- KS 检验：比较分布差异

- K L 散度、W a s s e r s t e i n 距离
- 概念漂移： $P(Y|X)$ 变化，需通过模型性能监控间接检测
- - -

九、特征存储与 Feature Store

9.1 Feature Store 的角色

Feature Store 是连接数据管道与模型训练 / 推理的中心化特征仓库，解决：

1. 训练 - 服务偏差：保证训练与推理使用相同的特征计算逻辑
2. 特征复用：跨团队、跨项目共享特征定义
3. 时间一致性：基于事件时间的点对点查询，避免未来信息泄露

9.2 核心组件

- 离线存储：批处理特征，用于训练 (P a r q u e t / I c e b e r g)
- 在线存储：低延迟服务，用于推理 (R e d i s / C a s s a n d r a)
- 特征注册中心：特征定义、版本、血缘
- 特征服务 A P I：统一检索接口

9.3 主流方案对比

方案 部署模式 特点

F e a s t 开源自托管 轻量、与主流框架集成好

H o p s w o r k s 开源 / 商业 集成完整 M L O p s 平台

T e c t o n 商业 S a a S 企业级、流批一体

A W S F e a t u r e S t o r e 云托管 与 S a g e M a k e r 深度集成

V e r t e x A I F e a t u r e S t o r e 云托管 与 G C P 生态集成

9.4 Feast 实践示例

```
from feast import FeatureStore, Entity, FeatureView
from feast.types import Float32, Int64
from datetime import timedelta
```



```

user = Entity(name="userid", joinkeys=["userid"])

userfeatures = FeatureView(
    name="userfeatures",
    entities=[user],
    schema=[
        Field(name="age", dtype=Int64),
        Field(name="avgclick7d", dtype=Float32),
        Field(name="conversionrate", dtype=Float32),
    ],
    online=True,
    source=batchsource,
    ttl=timedelta(days=30),
)

```

训练时获取历史特征（点对点）

```

store = FeatureStore(repopath=".")
trainingdf = store.gethistoricalfeatures(
    entitydf=entitydf, # 含 userid 与 eventtimestamp
    features=["userfeatures:age", "userfeatures:avgclick7d"],
).todf()

```

推理时在线读取

```

features = store.getonlinefeatures(
    features=["userfeatures:age", "userfeatures:avgclick7d"],
    entityrows=[{"userid": 1001}],
).todict()

- - -

```

十、关键词

数据工程、特征工程、特征编码、特征选择、数据清洗、Feature Store